Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase III · Secuencias y Transformer

Módulo 15

Construye tu GPT

Entrenamiento, evaluación y generación de un Transformer decoder

---

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 15: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
$\mathcal{V}$, $K = \lvert\mathcal{V}\rvert$	vocabulario y su tamaño
$u_{1:n}$	corpus tokenizado de n tokens
T, B	longitud de la ventana de contexto y tamaño del lote
$x_{1:T}$, $y_{1:T}$	ventana de entrada y ventana objetivo, con $y_t = x_{t+1}$
d, L, n_h, $d_k = d/n_h$	dimensión del modelo, número de bloques, de cabezas y dimensión por cabeza
$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$, $\boldsymbol{M}$	matrices de consultas, claves y valores, y máscara causal
$z \in \mathbb{R}^K$, $p = \mathrm{softmax}(z)$	logits y distribución predicha en una posición
ℓ, $\mathcal{L}$	pérdida de una posición y pérdida media
θ, N	parámetros del modelo y su número
s, S, W	paso de entrenamiento, número total de pasos y pasos de calentamiento
η_s, $\eta_{\max}$, $\eta_{\min}$	tasa de aprendizaje en el paso s y sus valores máximo y mínimo
λ, β_1, β_2	decaimiento de pesos y coeficientes de los momentos de AdamW
c, clip_c	umbral y operador de recorte del gradiente
τ, k, ρ	temperatura, parámetro de top-k y umbral de top-p
$H(p)$, $H(p,q)$, $D_{\mathrm{KL}}(p\Vert q)$	entropía, entropía cruzada y divergencia de Kullback–Leibler (en nats)

Índice

Sobre este resumen **ii**

1 Datos, ventanas y objetivo de entrenamiento **1**
1.1 El corpus como secuencia de tokens . . . 1
1.2 Ventanas de contexto y lotes . . . 1
1.3 La pérdida de entropía cruzada . . . 2
1.4 Gradiente y estabilidad numérica . . . 3

2 La arquitectura mini-GPT **4**
2.1 Definición del modelo . . . 4
2.2 La máscara causal . . . 5
2.3 Recuento de parámetros . . . 5
2.4 Del mini-GPT a los modelos abiertos . . . 6

3 Optimización: AdamW, calendario y recorte **7**
3.1 El paso de entrenamiento . . . 7
3.2 AdamW . . . 8
3.3 Calentamiento lineal y decaimiento coseno . . . 8
3.4 Recorte del gradiente por norma global . . . 9
3.5 El coste del entrenamiento . . . 9

4 Evaluación, puntos de control y curvas de pérdida **10**
4.1 Estimar la pérdida . . . 10
4.2 Puntos de control . . . 11
4.3 La cota inferior de la pérdida . . . 11
4.4 Lectura de las curvas de pérdida . . . 12

5 Generación e hiperparámetros **13**
5.1 Generación autorregresiva . . . 13
5.2 Temperatura . . . 13
5.3 Muestreo top-k y top-p . . . 14
5.4 Efecto de los hiperparámetros . . . 15

Bibliografía **16**

Tema 1

Datos, ventanas y objetivo de entrenamiento

Un modelo de lenguaje autorregresivo se entrena con una única secuencia larga de enteros, de la que se extraen ventanas de longitud fija. Este tema formaliza esa preparación de los datos, define la función que se minimiza (la entropía cruzada media sobre todas las posiciones de un lote) y demuestra tres propiedades de esa función que se usan durante todo el módulo: su gradiente respecto de los logits, su invariancia frente a traslaciones de los logits, que permite calcularla de forma estable, y su valor cuando la predicción es uniforme. Se sigue el §4.1 y el §5.1 de la tarea 1 de CS336 [11], el §9.3.3 de [16] y la clase 7 de CS224N [15].

1.1 El corpus como secuencia de tokens

Definición 1.1 (Corpus tokenizado). Sea $\mathcal{V}$ un vocabulario de tamaño K, identificado con $\{0, 1, \dots, K-1\}$. Un *corpus tokenizado* es una secuencia $u_{1:n} = (u_1, \dots, u_n) \in \mathcal{V}^n$. Si el texto original está formado por varios documentos, se concatenan en una sola secuencia, separados por un token delimitador.

La concatenación es una decisión de ingeniería con una justificación precisa: todas las ventanas tienen la misma longitud y no hace falta rellenar con símbolos de relleno (*padding*), que consumen cómputo sin aportar señal [11, §5.1]. El precio es que una ventana puede cruzar la frontera entre dos documentos; el delimitador permite al modelo aprender que lo que sigue no depende de lo anterior.

Un corpus sintético tiene una ventaja metodológica: se conoce el proceso que lo genera. Se sabe, por ejemplo, que tras «receta.» viene uno de seis verbos y que «un reloj» exige «negro» y no «negra». Esto permite juzgar qué ha aprendido el modelo y calcular la pérdida mínima alcanzable (tema 4).

1.2 Ventanas de contexto y lotes

Definición 1.3 (Ventana de contexto). Sea $T \geq 1$ la *longitud de contexto*. Para cada $i \in \{1, \dots, n-T\}$, la *ventana* que empieza en i es el par

$$x = (u_i, u_{i+1}, \dots, u_{i+T-1}), \qquad y = (u_{i+1}, u_{i+2}, \dots, u_{i+T}),$$

es decir, $y_t = x_{t+1}$ para $t < T$ e $y_T = u_{i+T}$. La secuencia y es la *ventana objetivo*.

Proposición 1.4. *Un corpus de n tokens tiene exactamente $n - T$ ventanas de longitud T, y cada una contiene T problemas de predicción: predecir y_t a partir del prefijo $x_{1:t}$, para $t = 1, \dots, T$.*

Definición 1.6 (Lote). Un *lote* de tamaño B es un par de matrices de enteros $X, Y \in \mathcal{V}^{B\times T}$ cuyas filas son B ventanas, con inicios elegidos de forma independiente y uniforme en $\{1, \dots, n-T\}$. Un lote contiene $B \cdot T$ predicciones.

Con $B = T = 32$, cada paso de entrenamiento ve 1024 predicciones; en 800 pasos son 819 200, unas 57 veces los 14 459 tokens de entrenamiento del ejemplo 1.2. Con tan pocos datos es inevitable repetir; los grandes modelos, en cambio, ven cada token una vez o pocas veces (módulo 16).

Observación 1.7 (Partición en entrenamiento y validación). El conjunto de validación se obtiene cortando el corpus *por posición*: el 90 % inicial (14 459 tokens) se usa para entrenar y el 10 % final (1 607 tokens) para validar. Si se eligieran ventanas al azar para cada conjunto, una ventana de validación podría solaparse con una de entrenamiento y la pérdida de validación subestimaría el error de generalización.

La alternativa al muestreo independiente de inicios es partir el corpus en trozos consecutivos sin solape, con un desplazamiento inicial aleatorio en cada época, como hace [16, §9.3.3]. El ejercicio 1.3 compara ambas estrategias.

1.3 La pérdida de entropía cruzada

En cada posición t de cada ventana, el modelo devuelve un vector de *logits* $z_t \in \mathbb{R}^K$ y la distribución predicha $p_t = \operatorname{softmax}(z_t)$, con $p_{t,j} = e^{z_{t,j}} / \sum_a e^{z_{t,a}}$.

Definición 1.8 (Pérdida de entrenamiento). La pérdida de una posición con objetivo $y \in \mathcal{V}$ y logits z es

$$\ell(z, y) = -\ln \operatorname{softmax}(z)_y.$$

La pérdida de un lote es la media sobre sus $B \cdot T$ posiciones,

$$\mathcal{L}(\theta) = \frac{1}{BT} \sum_{b=1}^{B} \sum_{t=1}^{T} \ell\big(z_{b,t}(\theta),\, Y_{b,t}\big),$$

donde $z_{b,t}(\theta)$ son los logits que el modelo de parámetros θ produce en la posición t de la fila b a partir de $X_{b,1:t}$.

La pérdida ℓ es la entropía cruzada entre la distribución degenerada en y y la distribución predicha [11, §4.1]. Su media sobre el corpus es la estimación de máxima verosimilitud del módulo 1 aplicada a un modelo con parámetros compartidos: minimizar $\mathcal{L}$ equivale a maximizar $\sum \ln P_\theta(y_t \mid x_{1:t})$.

Definición 1.9 (Perplejidad). Si una secuencia de m predicciones tiene pérdidas $\ell_1, \dots, \ell_m$, su *perplejidad* es

$$\mathrm{PPL} = \exp\Big(\frac{1}{m} \sum_{i=1}^{m} \ell_i\Big).$$

La perplejidad es el número de opciones equiprobables entre las que el modelo duda en media: un modelo uniforme sobre K símbolos tiene perplejidad K. Es independiente de la base del logaritmo, porque $e^{\mathcal{L}} = 2^{\mathcal{L}/\ln 2}$. En la práctica del módulo, el modelo de bigramas del módulo 1 llega a 2,035 nats por carácter (PPL = 7,65) y el mini-GPT a 0,324 (PPL = 1,38).

1.4 Gradiente y estabilidad numérica

Proposición 1.10 (Gradiente de la entropía cruzada). *Sea* $p = \mathrm{softmax}(z)$. *Para todo* $i \in \mathcal{V}$,

$$\frac{\partial \ell(z, y)}{\partial z_i} = p_i - \mathbb{1}[i = y],$$

es decir, $\nabla_z \ell = p - e_y$, *donde* e_y *es el vector* one-hot *de* y.

El gradiente de la proposición 1.10 es la única señal que recibe todo el mini-GPT: la retropropagación (módulo 10) la transporta desde los logits hasta los embeddings.

Proposición 1.12 (Invariancia frente a traslaciones). *Para todo* $c \in \mathbb{R}$, $\mathrm{softmax}(z + c\mathbf{1}) = \mathrm{softmax}(z)$. *En consecuencia, con* $z^\star = \max_a z_a$,

$$\ell(z, y) = -(z_y - z^\star) + \ln \sum_a e^{\,z_a - z^\star},$$

y todos los exponentes del segundo miembro son menores o iguales que 0.

La fórmula estable evita el desbordamiento: en precisión simple el mayor número representable es unos $3{,}4 \cdot 10^{38} \approx e^{88{,}7}$, de modo que e^{1000} daría infinito y el cociente, un valor indefinido. Es la técnica *log-sum-exp* del módulo 11, que CS336 exige en su implementación de la entropía cruzada [11, §4.1].

Proposición 1.13 (Pérdida de la predicción uniforme). *Si todos los logits de una posición son iguales, la pérdida vale* $\ln K$ *para cualquier objetivo.*

Con $K = 46$, $\ln 46 = 3{,}829$ nats (5,52 bits). Una red recién inicializada con pesos pequeños produce logits casi iguales, y su pérdida inicial está cerca de ese valor; en la práctica del módulo el primer lote da 4,046. Una pérdida inicial muy superior a $\ln K$ indica una inicialización defectuosa: el modelo está muy seguro de respuestas equivocadas.

Tema 2

La arquitectura mini-GPT

El mini-GPT es el bloque Transformer del módulo 14 apilado L veces entre una capa de entrada, que convierte tokens y posiciones en vectores, y una capa de salida, que devuelve un logit por token del vocabulario. Ninguna pieza es nueva; este tema fija la definición completa del modelo, demuestra la propiedad de causalidad que justifica entrenar las T predicciones de una ventana con una sola pasada y cuenta sus parámetros. La arquitectura es la de GPT-2 [10, §2.3], construida sobre el decoder de Vaswani *et al.* [13]; se sigue también el §3 de la tarea 1 de CS336 [11], la clase 5 de CS224N [15] y el §11.9.3 de [16].

2.1 Definición del modelo

Se recuerdan del módulo 11 la *normalización de capa* de un vector $v \in \mathbb{R}^d$,

$$\mathrm{LN}(v) = \gamma \odot \frac{v - \mu(v)\mathbf{1}}{\sqrt{\sigma^2(v) + \varepsilon}} + \beta, \qquad \mu(v) = \frac{1}{d}\sum_i v_i, \quad \sigma^2(v) = \frac{1}{d}\sum_i \big(v_i - \mu(v)\big)^2,$$

con parámetros $\gamma, \beta \in \mathbb{R}^d$, y del módulo 14 el perceptrón por posición $\mathrm{MLP}(v) = W_2\,\mathrm{GELU}(W_1 v + b_1) + b_2$, con $W_1 \in \mathbb{R}^{4d\times d}$ y $W_2 \in \mathbb{R}^{d\times 4d}$.

Definición 2.1 (Atención causal multicabeza). Sea $H \in \mathbb{R}^{T\times d}$ la matriz cuyas filas son las representaciones de las T posiciones, y sea n_h un divisor de d, con $d_k = d/n_h$. Se calculan $\boldsymbol{Q} = HW_Q$, $\boldsymbol{K} = HW_K$ y $\boldsymbol{V} = HW_V$, con $W_Q, W_K, W_V \in \mathbb{R}^{d\times d}$, y se parten por columnas en n_h bloques $\boldsymbol{Q}_r, \boldsymbol{K}_r, \boldsymbol{V}_r \in \mathbb{R}^{T\times d_k}$. La *máscara causal* es la matriz $\boldsymbol{M} \in \{0, -\infty\}^{T\times T}$ con $\boldsymbol{M}_{tj} = 0$ si $j \le t$ y $\boldsymbol{M}_{tj} = -\infty$ si $j > t$. Cada cabeza calcula

$$\mathrm{head}_r = \mathrm{softmax}\Big(\frac{\boldsymbol{Q}_r \boldsymbol{K}_r^\top}{\sqrt{d_k}} + \boldsymbol{M}\Big)\boldsymbol{V}_r, \qquad r = 1, \dots, n_h,$$

con la softmax aplicada por filas y el convenio $e^{-\infty} = 0$, y la salida es $\mathrm{MHA}(H) = [\mathrm{head}_1 \ \cdots \ \mathrm{head}_{n_h}]\, W_O + \mathbf{1} b_O^\top$, con $W_O \in \mathbb{R}^{d\times d}$ y $b_O \in \mathbb{R}^d$.

En la implementación, las tres proyecciones se agrupan en una sola matriz $d \times 3d$ sin sesgo, como sugiere CS336 [11, §3.4]; el número de parámetros es el mismo.

Definición 2.2 (Mini-GPT). Sean K, T, d, L y n_h enteros positivos con $n_h \mid d$. El *mini-GPT* con estos hiperparámetros transforma una ventana $x_{1:T} \in \mathcal{V}^T$ en logits $z_{1:T} \in \mathbb{R}^{T\times K}$ mediante:

(1) *entrada*: $h_t^0 = E_{x_t} + P_t$, donde $E \in \mathbb{R}^{K\times d}$ es la tabla de embeddings de token y $P \in \mathbb{R}^{T\times d}$ la de embeddings de posición, ambas aprendidas;

(2) *bloques*: para $\ell = 1, \dots, L$, con $H^{\ell-1}$ la matriz de filas $h_t^{\ell-1}$,

$$A^\ell = H^{\ell-1} + \mathrm{MHA}_\ell\big(\mathrm{LN}_{\ell,1}(H^{\ell-1})\big), \qquad H^\ell = A^\ell + \mathrm{MLP}_\ell\big(\mathrm{LN}_{\ell,2}(A^\ell)\big),$$

donde LN y MLP se aplican a cada fila por separado;

(3) *salida*: $z_t = W_{\mathrm{s}}\,\mathrm{LN}_{\mathrm{f}}(h_t^L) + b_{\mathrm{s}}$, con $W_{\mathrm{s}} \in \mathbb{R}^{K\times d}$ y $b_{\mathrm{s}} \in \mathbb{R}^K$.

La distribución predicha en la posición t es $p_t = \mathrm{softmax}(z_t)$.

La forma $x \leftarrow x + f(\mathrm{LN}(x))$ se llama *pre-norm*: la normalización se aplica a la entrada de cada subcapa y el flujo residual queda sin normalizar, lo que estabiliza el entrenamiento de redes profundas (módulo 11). GPT-2 introdujo exactamente estos dos cambios respecto de GPT: la normalización en la entrada de cada subbloque y una normalización final tras el último bloque [10, §2.3]. La figura 2.1 resume el flujo de datos. Suele decirse que la atención es la etapa en la que las posiciones se *comunican* y el MLP aquella en la que cada posición *procesa* lo recibido.

2.2 La máscara causal

Lema 2.3. *Sea $S \in \mathbb{R}^{T\times T}$ una matriz de puntuaciones. La fila t de* $\mathrm{softmax}(S + \boldsymbol{M})$ *vale*

$$a_{tj} = \begin{cases} \dfrac{e^{S_{tj}}}{\sum_{j'\le t} e^{S_{tj'}}} & j \le t, \\ 0 & j > t, \end{cases}$$

es decir, es la softmax de $(S_{t1}, \ldots, S_{tt})$ completada con ceros. En particular, cada fila suma 1 y la fila 1 es $(1, 0, \ldots, 0)$.

Teorema 2.5 (Causalidad). *En el mini-GPT, para cada $t \in \{1, \ldots, T\}$, los logits z_t dependen solo de $x_{1:t}$: si dos ventanas x y x' coinciden en las t primeras posiciones, entonces $z_t(x) = z_t(x')$.*

Corolario 2.6. *Una sola evaluación del mini-GPT sobre la ventana $x_{1:T}$ produce, en cada posición t, los mismos logits que se obtendrían evaluándolo sobre el prefijo $x_{1:t}$. Por tanto, una pasada calcula las T distribuciones $P_\theta(\cdot \mid x_{1:t})$ de la definición 1.8, cada una con su contexto correcto.*

Observación 2.7 (Fuga del futuro). Sin la máscara, la posición t puede atender a $x_{t+1} = y_t$. Una cabeza que copie el token siguiente lleva la pérdida a casi cero sin aprender a predecir nada, y el modelo falla al generar, cuando el futuro no existe. Una pérdida de validación próxima a cero en los primeros cientos de pasos es el síntoma característico (cuadro 4.2). La implementación original pone a $-\infty$ las entradas de la softmax correspondientes a conexiones ilegales [13, §3.2.3].

2.3 Recuento de parámetros

Proposición 2.8 (Parámetros del mini-GPT). *Con proyecciones W_Q, W_K, W_V sin sesgo, sesgos en W_O, en el MLP y en la cabeza, y sin compartir E con W_{s}, el mini-GPT de la definición 2.2 tiene*

$$N = \underbrace{Kd}_{E} + \underbrace{Td}_{P} + L\,(12d^2 + 10d) + \underbrace{2d}_{\mathrm{LN_f}} + \underbrace{dK + K}_{\textit{cabeza}}$$

parámetros. El número de cabezas n_h no interviene.

Cuadro 2.1: Parámetros del mini-GPT de la práctica: $K = 46$, $T = 32$, $d = 64$, $L = 2$, $n_h = 4$.

Pieza	Fórmula	Parámetros
Embedding de token	Kd	2 944
Embedding de posición	Td	2 048
Atención (por bloque)	$3d^2 + d^2 + d$	16 448
MLP (por bloque)	$8d^2 + 5d$	33 088
Dos normalizaciones (por bloque)	$4d$	256
Bloque completo	$12d^2 + 10d$	49 792
Dos bloques	$2\,(12d^2 + 10d)$	99 584
Normalización final	$2d$	128
Cabeza	$dK + K$	2 990
Total		107 694

El 92 % de los parámetros está en los bloques ($99\,584/107\,694 = 0{,}925$). El término $12d^2$ crece con el cuadrado de la anchura, mientras que las tablas crecen linealmente en d; en los modelos grandes la proporción de los bloques es aún mayor, salvo que el vocabulario sea muy grande (ejercicio 2.5). Es la misma cuenta que el módulo 14 hace para un modelo de $7 \cdot 10^9$ parámetros.

Observación 2.9 (Pesos atados). Vaswani *et al.* comparten la matriz de embeddings con la transformación lineal previa a la softmax [13, §3.4], de modo que $W_{\mathrm{s}} = E$ y se ahorran Kd parámetros. El mini-GPT de la práctica no los ata; con atadura tendría $107\,694 - 2\,944 = 104\,750$ parámetros.

2.4 Del mini-GPT a los modelos abiertos

Los modelos abiertos más citados usan la misma estructura con algunas piezas sustituidas. LLaMA normaliza la entrada de cada subcapa con RMSNorm, sustituye la GELU por SwiGLU con dimensión oculta $\frac{2}{3} \cdot 4d$ y elimina los embeddings de posición absolutos en favor de las rotaciones RoPE en cada capa [12, §2.2]; CS336 adopta esas mismas piezas [11, §3]. GPT-2, además de la normalización en la entrada de cada subbloque, escala por $1/\sqrt{N_{\mathrm{r}}}$ al inicializar los pesos de las capas residuales, con N_{r} el número de capas residuales, usa un vocabulario de 50 257 tokens y un contexto de 1024 [10, §2.3]. El cuadro 2.2 resume las diferencias.

Cuadro 2.2: Componentes del mini-GPT, de GPT-2 y de LLaMA, según los artículos citados y la implementación de GPT-2 de nanoGPT [6].

Componente	Mini-GPT	GPT-2 [10]	LLaMA [12]
Posiciones	tabla aprendida $T \times d$	tabla aprendida	RoPE en cada capa
Normalización	LayerNorm pre-norm	LayerNorm pre-norm y final	RMSNorm pre-norm
MLP	GELU, $d \to 4d \to d$	GELU, $d \to 4d \to d$	SwiGLU, dimensión $\frac{2}{3} \cdot 4d$
Tokenizador	caracteres, $K = 46$	BPE de bytes, $K = 50\,257$	BPE (SentencePiece)

Sobre los modelos comerciales, la información pública es limitada: el informe técnico de Gemini indica que sus modelos se construyen sobre decoders Transformer [3, §2]; xAI publicó en 2024 los pesos y el código de Grok-1, un Transformer de 64 capas con mezcla de 8 expertos y RoPE [14], y Anthropic no ha publicado la arquitectura de Claude.

Tema 3

Optimización: AdamW, calendario y recorte

Entrenar el mini-GPT consiste en repetir un paso muy breve: tomar un lote, calcular la pérdida, retropropagar y actualizar los parámetros. Lo que separa un entrenamiento estable de uno que diverge son tres piezas que rodean ese paso: el optimizador AdamW, el calendario de la tasa de aprendizaje y el recorte del gradiente. Este tema las define tal como las especifica la tarea 1 de CS336 [11, §4.2–4.5], demuestra sus propiedades principales y estima el coste del entrenamiento. AdamW y el calendario coseno se estudiaron en el módulo 7; aquí se usan en su forma de implementación, siguiendo a Loshchilov y Hutter [9, 8] y la receta documentada por LLaMA [12, §2.3].

3.1 El paso de entrenamiento

Definición 3.1 (Paso de entrenamiento). Dados los parámetros θ_s y el estado del optimizador en el paso $s = 0, 1, \ldots, S-1$, un *paso de entrenamiento* consiste en:

1. fijar la tasa η_s según el calendario;
2. extraer un lote (X, Y) (definición 1.6);
3. calcular los logits y la pérdida $\mathcal{L}(\theta_s)$ (definición 1.8);
4. poner a cero los gradientes acumulados y calcular $g = \nabla_\theta \mathcal{L}(\theta_s)$ por retropropagación;
5. recortar g por norma global (definición 3.9);
6. aplicar un paso de AdamW con tasa η_s (definición 3.2);
7. cada cierto número de pasos, evaluar la pérdida de entrenamiento y de validación y guardar un punto de control si la validación mejora (tema 4).

El orden importa. El recorte actúa sobre el gradiente ya calculado y antes de que el optimizador lo incorpore a sus momentos; si se aplicara después, un gradiente anómalo ya habría contaminado m y v. En PyTorch, el paso 4 son las llamadas `zero_grad` y `backward`, y el paso 6 es `step` [11, §4.2.1].

3.2 AdamW

Definición 3.2 (AdamW, forma de implementación). Sean $\beta_1, \beta_2 \in [0,1)$, $\lambda \geq 0$ y $\varepsilon > 0$, y sean $m_0 = v_0 = 0$. En la iteración $t = 1, 2, \ldots$, con tasa α y gradiente g_t, AdamW realiza

$$\begin{aligned} \alpha_t &= \alpha \frac{\sqrt{1-\beta_2^t}}{1-\beta_1^t}, & \theta &\leftarrow \theta - \alpha\lambda\theta, \\ m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t, & v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \\ \theta &\leftarrow \theta - \alpha_t \frac{m_t}{\sqrt{v_t} + \varepsilon}, \end{aligned}$$

donde el cuadrado, la raíz y el cociente se aplican componente a componente.

Es el algoritmo 1 de CS336 [11, §4.3], que reproduce el algoritmo 2 de Loshchilov y Hutter [9]. La corrección de sesgo de los momentos del módulo 7 aparece aquí agrupada en α_t; la diferencia con la forma $\hat{m}_t/(\sqrt{\hat{v}_t}+\varepsilon)$ es solo la escala efectiva de ε. El término $-\alpha\lambda\theta$ es el *decaimiento de pesos desacoplado*: no pasa por los momentos y encoge todos los pesos en la misma proporción.

Proposición 3.3 (Encogimiento por decaimiento). *Si un parámetro recibe gradiente nulo durante los pasos $s = 0, \ldots, S-1$, con tasas η_s y $\lambda\eta_s < 1$, su valor final es*

$$\theta_S = \theta_0 \prod_{s=0}^{S-1} (1 - \lambda\eta_s) \;\leq\; \theta_0 \exp\Big(-\lambda \sum_{s=0}^{S-1} \eta_s\Big) \qquad (\theta_0 \geq 0).$$

Los momentos m y v forman el *estado* del optimizador: dos números por parámetro, además del contador t. Por eso el estado ocupa el doble que el modelo y debe guardarse en los puntos de control (tema 4). LLaMA usa $\beta_1 = 0{,}9$, $\beta_2 = 0{,}95$ y $\lambda = 0{,}1$ [12, §2.3]; CS336 señala que (0,9; 0,999) es la elección típica y (0,9; 0,95) la habitual en modelos de lenguaje grandes [11, §4.3].

3.3 Calentamiento lineal y decaimiento coseno

Definición 3.5 (Calendario del módulo). Sean $0 \leq \eta_{\min} < \eta_{\max}$, $W \geq 1$ pasos de calentamiento y $S > W$ pasos totales. Para $s = 0, \ldots, S-1$,

$$\eta_s = \begin{cases} \eta_{\max} \dfrac{s+1}{W}, & s < W, \\[2ex] \eta_{\min} + \tfrac{1}{2}(\eta_{\max} - \eta_{\min})\Big(1 + \cos\dfrac{\pi(s-W)}{S-W}\Big), & s \geq W. \end{cases}$$

El tramo coseno es la ecuación (5) de SGDR [8, §3] con un único ciclo. La práctica usa $\eta_{\min} = \eta_{\max}/10$, como LLaMA [12, §2.3] y la configuración de ejemplo de nanoGPT [6].

Proposición 3.6 (Propiedades del calendario). *En el calendario de la definición 3.5:*

(i) η_s crece estrictamente en $\{0, \ldots, W-1\}$, con $\eta_0 = \eta_{\max}/W$ y $\eta_{W-1} = \eta_W = \eta_{\max}$;

(ii) η_s decrece estrictamente en $\{W, \ldots, S-1\}$, y la expresión del tramo coseno vale $\eta_{\min}$ en $s = S$;

(iii) en el punto medio del tramo coseno, $s = W + (S-W)/2$, vale $(\eta_{\max} + \eta_{\min})/2$;

(iv) para todo $u \in [0, S-W]$, $\eta(W+u) + \eta(S-u) = \eta_{\max} + \eta_{\min}$, *donde* $\eta(\cdot)$ *denota la expresión del tramo coseno evaluada en un real.*

Observación 3.8 (El convenio de CS336). CS336 define el calentamiento como $\alpha_t = (t/T_w)\,\alpha_{\max}$ para $t < T_w$, el coseno entre T_w y T_c y una tasa constante $\alpha_{\min}$ para $t > T_c$ [11, §4.4]. Con ese convenio, $\alpha_0 = 0$ (el primer paso no mueve los pesos, salvo por el decaimiento, que también se multiplica por α) y el pico se alcanza en $t = T_w$; con el del módulo, la tasa empieza en $\eta_{\max}/W$ y llega al pico en $s = W - 1$. Ambas formas difieren en un paso y son equivalentes en la práctica.

La justificación de ambas fases se dio en el módulo 7. Durante el calentamiento, los momentos de Adam se han estimado con muy pocos gradientes, y estos son grandes e irregulares; una tasa grande desde el primer paso puede llevar a una región de la que el entrenamiento no se recupera. Al final, una tasa pequeña reduce el exceso de pérdida debido al ruido del gradiente estocástico. LLaMA documenta 2000 pasos de calentamiento y decaimiento coseno hasta el 10 % de la tasa máxima [12, §2.3].

3.4 Recorte del gradiente por norma global

Definición 3.9 (Recorte por norma global). Sea $g = (g^{(1)}, \dots, g^{(P)})$ el gradiente de todos los tensores de parámetros, considerado como un único vector, y $\|g\|_2 = \left(\sum_q \|g^{(q)}\|_2^2\right)^{1/2}$ su *norma global*. Dados $c > 0$ y $\varepsilon > 0$ pequeño,

$$\operatorname{clip}_c(g) = \begin{cases} g, & \|g\|_2 < c, \\ \dfrac{c}{\|g\|_2 + \varepsilon}\, g, & \|g\|_2 \geq c. \end{cases}$$

Es la definición de CS336, con $\varepsilon = 10^{-6}$ [11, §4.5]; LLaMA usa $c = 1$ [12, §2.3], como la práctica del módulo. El recorte protege de los gradientes anómalos que aparecen cuando la superficie de pérdida tiene acantilados [4, §8.2.4], y que de otro modo dañarían durante muchos pasos las medias móviles de Adam.

Proposición 3.10 (Propiedades del recorte). *Para todo* g*:*

(i) $\operatorname{clip}_c(g)$ *es un múltiplo positivo de* g*, luego tiene su misma dirección y sentido;*

(ii) $\|\operatorname{clip}_c(g)\|_2 < c$ *si* $\|g\|_2 \geq c$*, y* $\operatorname{clip}_c(g) = g$ *en otro caso;*

(iii) recortar cada tensor por separado con el mismo umbral produce, en general, un vector que no es múltiplo de g*.*

El recorte global no cambia la dirección de descenso; solo limita la longitud del paso. El recorte por tensor, en cambio, altera el peso relativo de cada parte del modelo en la actualización.

3.5 El coste del entrenamiento

Por la regla del módulo 14, entrenar un modelo de N parámetros con D tokens cuesta $C \approx 6ND$ operaciones de coma flotante (FLOP): $2ND$ en la pasada hacia delante y el doble en la retropropagación. CS336 usa la misma hipótesis sobre la retropropagación [11, §4.3, problema *adamw_accounting*].

Tema 4

Evaluación, puntos de control y curvas de pérdida

La pérdida de entrenamiento de un lote es una cantidad ruidosa que, además, no mide lo que interesa: cómo se comporta el modelo con texto que no ha visto. Este tema define el estimador de la pérdida que se usa para dibujar las curvas, describe qué debe contener un punto de control para poder reanudar el entrenamiento sin perturbarlo, demuestra que la entropía del proceso que genera el texto es una cota inferior de la pérdida esperada y presenta una guía de diagnóstico a partir de la forma de las curvas. Se sigue el §7.8 y el capítulo 11 de [4], los §5.2 y §7.2.3 de la tarea 1 de CS336 [11] y el capítulo 2 de Cover y Thomas [1].

4.1 Estimar la pérdida

Definición 4.1 (Pérdida estimada). Sean $(X^{(1)}, Y^{(1)}), \dots, (X^{(r)}, Y^{(r)})$ lotes extraídos de un conjunto de datos (entrenamiento o validación). La *pérdida estimada* del modelo de parámetros θ es

$$\hat{\mathcal{L}}(\theta) = \frac{1}{r} \sum_{q=1}^{r} \mathcal{L}_q(\theta),$$

donde $\mathcal{L}_q$ es la pérdida media del lote q (definición 1.8). En la práctica del módulo, $r = 20$ y los lotes se extraen con un generador de semilla fija, de modo que son los mismos en todas las evaluaciones.

Si los lotes se extraen de forma independiente, $\hat{\mathcal{L}}(\theta)$ es un estimador insesgado de la pérdida esperada y su varianza es la de un lote dividida por r. Usar siempre los mismos lotes tiene una ventaja adicional cuando se comparan dos puntos de la curva.

Proposición 4.2 (Lotes comunes). *Sean θ_a y θ_b dos estados del modelo, y sean $\hat{\mathcal{L}}_a$ y $\hat{\mathcal{L}}_b$ sus pérdidas estimadas, vistas como variables aleatorias a través de la elección de los lotes. Entonces*

$$\operatorname{Var}\big(\hat{\mathcal{L}}_a - \hat{\mathcal{L}}_b\big) = \operatorname{Var}(\hat{\mathcal{L}}_a) + \operatorname{Var}(\hat{\mathcal{L}}_b) - 2\operatorname{Cov}(\hat{\mathcal{L}}_a, \hat{\mathcal{L}}_b).$$

Si los dos estimadores usan lotes independientes, la covarianza es nula; si usan los mismos lotes y la covarianza es positiva, la varianza de la diferencia es menor.

Dos estados próximos del modelo encuentran difíciles los mismos lotes, de modo que sus pérdidas sobre lotes comunes están positivamente correlacionadas; la diferencia entre dos puntos de la curva refleja entonces sobre todo el cambio del modelo y no el azar del muestreo. Es la técnica de los *números aleatorios comunes* de la simulación estocástica.

Durante la evaluación se usan dos instrucciones de PyTorch: `model.eval()`, que desactiva los componentes que se comportan distinto al entrenar (como el *dropout*), y el contexto

`torch.no_grad()`, que evita construir el grafo de cómputo y ahorra memoria y tiempo. Al terminar, `model.train()` devuelve el modelo al modo de entrenamiento.

4.2 Puntos de control

Definición 4.3 (Punto de control). Un *punto de control* es un registro que contiene todo lo necesario para reanudar el entrenamiento como si no se hubiera interrumpido: los parámetros θ, el estado del optimizador (en AdamW, los momentos m, v y el contador de pasos) y el número de paso s, que fija la tasa del calendario [11, §5.2]. Es habitual añadir la pérdida de validación y la configuración del modelo.

Si se reanuda sin el estado del optimizador, AdamW reinicia $m = v = 0$ y $t = 0$: el primer paso usa momentos estimados con un solo gradiente, sin la protección de un calentamiento, y la curva de pérdida da un salto. Si se reanuda sin el número de paso, el calendario vuelve a empezar.

Proposición 4.4 (Memoria del estado de entrenamiento). *En precisión simple (4 bytes por número), los parámetros, los gradientes y los dos momentos de AdamW de un modelo de N parámetros ocupan $16N$ bytes, sin contar las activaciones.*

Para el mini-GPT son $107\,694 \cdot 16 = 1{,}72$ MB, de los que el modelo ocupa 0,43 MB. Para un modelo de $7 \cdot 10^9$ parámetros, la misma cuenta da 112 GB, más que la memoria de una GPU de 80 GB; por eso existen las técnicas de partición del estado del optimizador del módulo 17.

Observación 4.5 (Parada temprana). Guardar el punto de control solo cuando la pérdida de validación mejora y devolver al final el mejor es el algoritmo de *parada temprana* de Goodfellow, Bengio y Courville [4, §7.8], que actúa como regularizador: limita el número efectivo de pasos de optimización. Los grandes entrenamientos guardan además puntos de control frecuentes porque, con miles de GPU, las interrupciones son rutinarias: el informe de Llama 3 documenta 466 interrupciones en un periodo de 54 días de preentrenamiento [7, §3.3.4].

4.3 La cota inferior de la pérdida

Teorema 4.6 (La entropía como cota inferior). *Sean p y q distribuciones sobre $\mathcal{V}$ con $q(x) > 0$ siempre que $p(x) > 0$. Entonces*

$$H(p, q) = -\sum_{x} p(x) \ln q(x) = H(p) + D_{\mathrm{KL}}(p \| q) \;\geq\; H(p),$$

con igualdad si y solo si $q = p$.

Si p es la distribución real del token siguiente y q la del modelo, la pérdida esperada no puede bajar de $H(p)$. El ejercicio 4.6 extiende el resultado a distribuciones condicionadas al contexto, que es el caso de un modelo de lenguaje. En consecuencia, si la pérdida media de validación sale claramente por debajo de la entropía del proceso generador, hay una fuga de información. En texto humano no se conoce H, pero el mismo resultado explica por qué las curvas de los grandes modelos se aplanan: las leyes de escalado del módulo 16 incluyen un término constante que se interpreta como la entropía irreducible del texto.

4.4 Lectura de las curvas de pérdida

Una curva de pérdida se interpreta respecto de tres referencias horizontales: $\ln K$, la pérdida de la predicción uniforme (proposición 1.13); la pérdida del modelo de bigramas, que es lo mínimo que hay que superar; y, cuando se conoce, la entropía H del texto, que no puede superarse (teorema 4.6). El cuadro 4.1 recoge la curva de la práctica.

Cuadro 4.1: Pérdidas del mini-GPT base de la práctica, en nats por carácter. Referencias: $\ln 46 = 3{,}829$; bigramas, 2,035; entropía del corpus, $\approx 0{,}125$.

Momento	Entrenamiento	Validación	Lectura
Paso 0	4,046 (un lote)	—	cerca de $\ln K$: predicción casi uniforme
Paso 100	2,017	2,046	alcanza el nivel del bigrama
Paso 200	1,040	1,154	aprende palabras y plantillas
Paso 400	0,370	0,425	usa el contexto largo
Paso 800	0,264	0,324	afinado final con tasa baja

La forma. Hay una caída rápida hasta el nivel del bigrama (pasos 0–100), en la que el modelo aprende las frecuencias de pares de caracteres, y una segunda bajada larga (100–500), en la que aprende a usar el contexto: qué plantilla está escribiendo y qué palabras han aparecido ya. La pendiente final se suaviza a medida que el coseno reduce la tasa.

La brecha. Al final, la diferencia entre validación y entrenamiento es 0,06 nats. Es un sobreajuste leve: el modelo ha memorizado algunas frases, lo que es esperable cuando cada token se ve unas 57 veces. Si la pérdida de validación empezara a subir mientras la de entrenamiento baja, habría que parar, usar más datos o regularizar más.

Cuadro 4.2: Síntomas frecuentes en las curvas de pérdida y su diagnóstico probable.

Síntoma	Diagnóstico probable	Qué probar
No baja de $\ln K$	error de programación: objetivos desalineados, tasa nula	sobreajustar un solo lote
Baja y después da NaN	tasa demasiado alta	bajar $\eta_{\max}$, alargar el calentamiento, recortar
Baja muy despacio	tasa demasiado baja	subir $\eta_{\max}$
Validación sube y entrenamiento baja	sobreajuste	más datos, decaimiento de pesos, parar antes
Pérdida casi nula en ambas	fuga del futuro	revisar la máscara causal (observación 2.7)

La primera prueba de la tercera columna es la estrategia de depuración más recomendada: un modelo correcto debe poder llevar a casi cero la pérdida de un conjunto de datos diminuto, como un único lote; si no lo consigue, el fallo está en el software y no en la capacidad del modelo [4, §11.5], [11, §7.2.3]. La misma sección de [4] recomienda razonar con los errores de entrenamiento y de prueba: error bajo en entrenamiento y alto en prueba indica sobreajuste; error alto en ambos deja abiertas la falta de capacidad y un defecto del programa.

Tema 5

Generación e hiperparámetros

Generar texto es aplicar la regla de la cadena del módulo 1 con el modelo como calculadora de probabilidades condicionadas: se predice la distribución del token siguiente, se muestrea, se añade el token al contexto y se repite. Este tema formaliza el bucle de generación, estudia las tres transformaciones de la distribución que exponen las interfaces de los modelos actuales (temperatura, top-k y top-p) y demuestra que la entropía de la distribución crece con la temperatura. Termina con el efecto de los hiperparámetros sobre el entrenamiento del mini-GPT. Se sigue el §6 de la tarea 1 de CS336 [11], el §9.5.5 de [16], los artículos de Fan, Lewis y Dauphin [2] y de Holtzman *et al.* [5], y el §11.4 de [4].

5.1 Generación autorregresiva

Definición 5.1 (Bucle de generación). Sea un modelo con contexto máximo T, una secuencia inicial (*prompt*) $w_{1:m}$, un número máximo de tokens nuevos G y una transformación Φ que convierte unos logits en una distribución. Para $g = 1, \dots, G$:

1. recortar el contexto a sus últimos T tokens, $c = w_{\max(1,\, m+g-T):\, m+g-1}$;
2. evaluar el modelo sobre c y tomar los logits z de la última posición;
3. muestrear $w_{m+g} \sim \Phi(z)$ y añadirlo a la secuencia;
4. detenerse si w_{m+g} es el token delimitador.

Por el corolario 2.6, los logits de la última posición son los de la distribución $P_\theta(\cdot \mid c)$. Con $\Phi = \mathrm{softmax}$ se muestrea del propio modelo [11, §6, ecuaciones (21)–(22)]; si en cada paso se elige el token más probable, el procedimiento es la *decodificación voraz* [16, §9.5.5]. Cada paso exige una pasada completa del modelo sobre el contexto; el módulo 19 estudia cómo evitar recalcular lo ya calculado con la caché de claves y valores.

5.2 Temperatura

Definición 5.2 (Softmax con temperatura). Para $\tau > 0$, $\mathrm{softmax}(z, \tau)_i = e^{z_i/\tau} \big/ \sum_j e^{z_j/\tau}$. En este módulo la temperatura se escribe τ para no confundirla con la longitud de contexto T.

Proposición 5.3 (Límites). *Sea $z \in \mathbb{R}^K$ con un único máximo en el índice $i^\star$. Cuando $\tau \to 0^+$, $\mathrm{softmax}(z, \tau) \to e_{i^\star}$; cuando $\tau \to \infty$, $\mathrm{softmax}(z, \tau) \to (1/K, \dots, 1/K)$.*

Teorema 5.4 (La entropía crece con la temperatura). *Sea $z \in \mathbb{R}^K$ no constante y, para $b > 0$, $p^{(b)} = \mathrm{softmax}(bz)$, de modo que $b = 1/\tau$. Entonces*

$$\frac{d}{db}\, H\big(p^{(b)}\big) = -b\, \mathrm{Var}_{p^{(b)}}(z) < 0,$$

donde $\operatorname{Var}_p(z) = \sum_i p_i z_i^2 - \big(\sum_i p_i z_i\big)^2$. *En consecuencia,* $H\big(\operatorname{softmax}(z,\tau)\big)$ *es estrictamente creciente en* τ.

La temperatura es la misma que la de la distribución de Boltzmann de la física estadística (módulo 1). Holtzman *et al.* observan que bajar la temperatura mejora la calidad del texto generado a costa de reducir su diversidad [5, §3.3].

5.3 Muestreo top-k y top-p

Definición 5.6 (Top-k). Sea $1 \le k \le K$ y sea $I_k(z)$ el conjunto de los índices de los k mayores logits, con los empates resueltos por el índice menor. El *muestreo top-k* usa la distribución $\operatorname{softmax}\big(\operatorname{top}_k(z)\big)$, donde $\operatorname{top}_k(z)_i = z_i$ si $i \in I_k(z)$ y $-\infty$ en otro caso.

Fan, Lewis y Dauphin introdujeron este procedimiento con $k = 10$ para generar relatos y observaron que producía textos menos repetitivos que la búsqueda en haz [2, §5.4]. Con $k = 1$ el muestreo coincide con la decodificación voraz; con $k = K$ no cambia nada. Con $z = (2;\ 1;\ 0{,}5;\ -1)$, $k = 2$ y $\tau = 1$, la distribución pasa a ser $(0{,}731;\ 0{,}269;\ 0;\ 0)$: desaparece la cola de tokens improbables, que con 46 caracteres puede acumular una masa apreciable.

Definición 5.7 (Muestreo de núcleo o top-p). Sea q una distribución sobre $\mathcal{V}$ y $\rho \in (0, 1]$. Sea $V(\rho)$ el menor conjunto de índices tal que $\sum_{j \in V(\rho)} q_j \ge \rho$. El *muestreo de núcleo* usa la distribución

$$\tilde{q}_i = \begin{cases} q_i \big/ \sum_{j \in V(\rho)} q_j, & i \in V(\rho), \\ 0, & \text{en otro caso.} \end{cases}$$

Es la definición de Holtzman *et al.* [5, §3.1, ecuaciones (2) y (3)], reproducida por CS336 [11, §6, ecuación (24)], donde el umbral se denota p. La distribución q suele ser ya la softmax con temperatura.

Proposición 5.8 (Cálculo del núcleo). *Sean* $q_{(1)} \ge q_{(2)} \ge \cdots \ge q_{(K)}$ *las probabilidades ordenadas y* $m = \min\{j : \sum_{i \le j} q_{(i)} \ge \rho\}$. *Todo conjunto de masa al menos* ρ *tiene al menos* m *elementos, y el conjunto* S_m *de los* m *índices más probables alcanza esa masa. Por tanto, los conjuntos mínimos tienen exactamente* m *elementos y* S_m *es, entre ellos, uno de masa máxima.*

Observación 5.9. El conjunto mínimo no es único en general, aunque no haya empates: con $q = (0{,}5;\ 0{,}3;\ 0{,}2)$ y $\rho = 0{,}6$, tanto $\{1, 2\}$ como $\{1, 3\}$ tienen dos elementos y masa al menos 0,6. Por convenio, $V(\rho)$ designa el conjunto S_m, que es el que se obtiene al ordenar la distribución y acumular probabilidades hasta alcanzar ρ, como prescribe CS336 [11, §6].

La ventaja de top-p sobre top-k es que el número de candidatos se adapta a la forma de la distribución. Con una distribución plana, un k pequeño elimina continuaciones razonables; con una distribución muy concentrada, un k grande deja entrar candidatos inapropiados, cuya probabilidad crece al renormalizar [5, §3.2]. Las interfaces de programación de modelos comerciales como Claude y Gemini exponen públicamente los parámetros de temperatura, top-k y top-p.

Muestras del mini-GPT. Con $\tau = 0{,}8$ y $k = 10$, casi todas las líneas generadas son correctas:

```
objeto. una lámpara azul de cristal. sirve para esperar sentado.
objeto. una caja azul de tela. sirve para leer de noche.
receta. hornea verde paperado.
```

La tercera muestra el límite del modelo: mezcla un color propio de los objetos en una receta. Con $\tau = 3$ la salida se degrada a símbolos sueltos, porque la distribución se acerca a la uniforme y los dígitos y signos aparecen en cualquier lugar.

5.4 Efecto de los hiperparámetros

El cuadro 5.1 resume entrenamientos de 800 pasos sobre el mismo corpus y con la misma semilla, en los que solo cambia lo indicado.

Cuadro 5.1: Efecto de los hiperparámetros en el mini-GPT (resultados del cuaderno de prácticas). Base: $d = 64$, $L = 2$, $n_h = 4$, $T = 32$, $\eta_{\max} = 3 \cdot 10^{-3}$. Pérdidas de validación en nats por carácter; tiempo relativo al modelo base.

Cambio	N	Paso 100	Final	Tiempo	Lectura
Base	107 694	2,046	0,324	1	referencia
$\eta_{\max} = 3 \cdot 10^{-4}$	107 694	2,959	1,840	1	demasiado lenta
$\eta_{\max} = 3 \cdot 10^{-2}$	107 694	1,304	0,297	1	mejor: tolera tasas altas
ídem, sin calentamiento	107 694	1,232	0,306	1	algo peor
$\eta_{\max} = 3 \cdot 10^{-1}$	107 694	2,863	2,244	1	inestable
Tasa constante	107 694	2,046	0,337	1	falta el afinado final
$d = 32$	29 294	2,308	0,649	0,5	poca capacidad
$L = 1$	57 902	2,128	0,392	0,4	un bloque no basta
$n_h = 1$	107 694	2,062	0,336	0,7	algo peor
$T = 8$	106 158	2,068	0,694	0,3	no ve la etiqueta
$d = 128$, $L = 3$	609 838	1,441	0,297	2,4	mejor, más caro

La tasa de aprendizaje es el factor dominante: entre $3 \cdot 10^{-4}$ y $3 \cdot 10^{-2}$ la pérdida final pasa de 1,840 a 0,297, y con $3 \cdot 10^{-1}$ el entrenamiento se rompe. Goodfellow, Bengio y Courville la señalan como el hiperparámetro más importante si solo se puede ajustar uno [4, §11.4.1], y CS336 pide localizar empíricamente la tasa a partir de la cual el entrenamiento diverge y relacionarla con la mejor [11, §7.2, problema *learning_rate*]. El contexto es el segundo factor: con $T = 8$ la pérdida se duplica, porque el modelo pierde la información que más reduce la incertidumbre, el tipo de línea (ejercicio 5.6). Multiplicar por 5,7 los parámetros da la misma mejora que subir la tasa a $3 \cdot 10^{-2}$ sin coste adicional. El calentamiento y el coseno aportan poco en este modelo (unas centésimas de nat cada uno), porque es pequeño y el recorte ya lo protege; en los modelos grandes son imprescindibles.

Observación 5.10 (Reglas prácticas). De lo anterior se desprenden cuatro reglas: (i) ajustar primero la tasa de aprendizaje, barriendo potencias de 10 o de $\sqrt{10}$, y quedarse con la mayor que sea estable; (ii) antes de escalar, comprobar que el modelo sobreajusta un único lote hasta pérdida casi nula; (iii) elegir T de modo que cubra la dependencia que se quiere capturar, aquí una línea completa de unos 60 a 80 caracteres; (iv) más parámetros solo ayudan si hay datos y pasos suficientes para entrenarlos, como precisan las leyes de escalado del módulo 16.

Bibliografía

[1] T. M. Cover y J. A. Thomas. *Elements of Information Theory*, 2.ª ed. Wiley, 2006. Cap. 2, «Entropy, Relative Entropy, and Mutual Information».

[2] A. Fan, M. Lewis e Y. Dauphin. Hierarchical neural story generation. En *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018. https://arxiv.org/abs/1805.04833

[3] Gemini Team, Google. Gemini: A family of highly capable multimodal models. Informe técnico, 2023. https://arxiv.org/abs/2312.11805

[4] I. Goodfellow, Y. Bengio y A. Courville. *Deep Learning*. MIT Press, 2016. Cap. 7, «Regularization for Deep Learning»; cap. 8, «Optimization for Training Deep Models»; cap. 11, «Practical Methodology». https://www.deeplearningbook.org/

[5] A. Holtzman, J. Buys, L. Du, M. Forbes e Y. Choi. The curious case of neural text degeneration. En *International Conference on Learning Representations (ICLR)*, 2020. https://arxiv.org/abs/1904.09751

[6] A. Karpathy. nanoGPT. Repositorio de software, GitHub, 2022–2025. Archivos `model.py`, `train.py`, `sample.py` y `config/train_shakespeare_char.py`. Desde noviembre de 2025 su autor lo declara obsoleto y remite a nanochat. https://github.com/karpathy/nanoGPT

[7] Llama Team, AI @ Meta. The Llama 3 herd of models. Informe técnico, 2024. https://arxiv.org/abs/2407.21783

[8] I. Loshchilov y F. Hutter. SGDR: Stochastic gradient descent with warm restarts. En *International Conference on Learning Representations (ICLR)*, 2017. https://arxiv.org/abs/1608.03983

[9] I. Loshchilov y F. Hutter. Decoupled weight decay regularization. En *International Conference on Learning Representations (ICLR)*, 2019. https://arxiv.org/abs/1711.05101

[10] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei e I. Sutskever. Language models are unsupervised multitask learners. Informe técnico, OpenAI, 2019.

[11] Stanford CS336. Language Modeling from Scratch. Stanford University, primavera de 2026. Tarea 1 (*basics*), «Building a Transformer LM», versión 26.0.3. https://github.com/stanford-cs336/assignment1-basics

[12] H. Touvron *et al.* LLaMA: Open and efficient foundation language models. Prepublicación, 2023. https://arxiv.org/abs/2302.13971

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser e I. Polosukhin. Attention is all you need. En *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017. https://arxiv.org/abs/1706.03762

[14] xAI. Grok-1: pesos abiertos y código de ejemplo. Repositorio de software, GitHub, 2024. https://github.com/xai-org/grok-1

[15] D. Yang y Y. Choi. CS224N: Natural Language Processing with Deep Learning. Stanford University, invierno de 2026. Clase 5, «Transformers»; clase 7, «Pretraining (Scaling, Systems, Data)». https://web.stanford.edu/class/cs224n/

[16] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. Cap. 9, «Recurrent Neural Networks» (§9.3 y §9.5); cap. 11, «Attention Mechanisms and Transformers» (§11.9). https://d2l.ai/